

# Database Trigger

Presented by

**Dr. Md. Abir Hossain**

Associate Professor

Dept. of ICT

MBSTU

# Trigger

- A trigger in MySQL is a set of SQL statements that reside in a system catalog.
- It is a **special type of stored procedure** that is invoked automatically in response to an event.
- Each trigger is associated with a table, which is activated on any **DML** statement such as **INSERT, UPDATE, or DELETE**.
- Triggers are of two types according to the SQL standard:
  - **Row-level triggers** and
  - **Statement-level triggers**.
- The trigger code block is executed either before or after each row is affected by the triggering statement.

# Types of Trigger

- **Row-Level Trigger**
- This trigger is activated **for each row by a triggering statement** such as insert, update, or delete.
- For example, if a table has inserted, updated, or deleted multiple rows, the row trigger is fired automatically for each row affected by the insert, update, or delete statement.
- More precisely, if deletion is defined as a triggering event for a particular table, and a single DELETE statement deletes five rows from that table, the trigger fires five times, once for each row.

# Types of Trigger

- **Statement-Level Trigger**

- It is a trigger, which is fired once for each event that occurs on a table regardless of how many rows are inserted, updated, or deleted.
- More precisely, if deletion is defined as a triggering event for a particular table, and a single DELETE statement deletes five rows from that table, the trigger fires once.
- A typical representation of trigger is presented below:

```
Create Trigger TriggerNAME
```

```
[BEFORE/AFTER] [INSERT/DELETE/UPDATE] ON TableName FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.bill >= 500 THEN SET NEW.customer_status = "Rich";
```

```
END IF;
```

```
END //
```

# Why we need/use triggers in MySQL?

We need to use triggers in MySQL due to the following features:

- Triggers help us to enforce business rules.
- Triggers help us to validate data even before they are inserted or updated.
- Triggers help us to keep a log of records like maintaining audit trails in tables.
- SQL triggers provide an alternative way to check the integrity of data.
- Triggers provide an alternative way to run the scheduled task.
- Triggers increases the performance of SQL queries because it does not need to compile each time the query is executed.
- Triggers reduce the client-side code that saves time and effort.
- Triggers help us to scale our application across different platforms.
- Triggers are easy to maintain.

# Limitations of Using Triggers in MySQL

There are few limitations of using triggers in MySQL:

- MySQL triggers do not allow to use of all type of validations.
  - For example, we can't use the NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints for simple validations.
- Trigger-oriented troubleshooting is difficult because triggers are invoked and executed invisibly from the client application.
- Triggers may increase the overhead of the database server.

# Trigger Types based on Action or event

- We can define the maximum six types of actions or events in the form of triggers:
  - Before Insert: It is activated before the insertion of data into the table.
  - After Insert: It is activated after the insertion of data into the table.
  - Before Update: It is activated before the update of data in the table.
  - After Update: It is activated after the update of the data in the table.
  - Before Delete: It is activated before the data is removed from the table.
  - After Delete: It is activated after the deletion of data from the table.

# Before Insert Trigger

- It is activated before the insertion of data into the table.
- Consider the customer table

| customerNo | customerName   | purItem   | bill | contactAddress  | branch         | customer_status |
|------------|----------------|-----------|------|-----------------|----------------|-----------------|
| cus-20002  | kabir          | lungi     | 500  | NULL            | swopno,tangail | NULL            |
| cus-20003  | akbar          | towel     | 200  | dhanmodi,dhaka  | swopno,tangail | NULL            |
| cus-20004  | Rahman         | noodles   | 200  | dhanmodi,dhaka  | Tangail        | NULL            |
| cus-20004  | Rahman         | puff rice | 100  | dhanmodi,dhaka  | Tangail        | NULL            |
| cus-20003  | akbar          | dettol    | 60   | dhanmodi,dhaka  | swopno,tangail | NULL            |
| cus-20005  | rabir          | chips     | 50   | NULL            | swopno,tangail | NULL            |
| cus-20001  | obaaidul kader | mafler    | 1000 | dhanmondi,dhaka | swopno,tangail | NULL            |
| cus-20006  | musa           | punjabi   | 750  | NULL            | NULL           | Rich            |
| cus-20007  | Ornob          | music     | NULL | NULL            | NULL           | NULL            |
| cus-20007  | lily           | NULL      | 600  | NULL            | NULL           | Rich            |
| cus-20007  | lily           | NULL      | 600  | NULL            | NULL           | Rich            |
| cus-20008  | ritica         | NULL      | 333  | NULL            | NULL           | Middle_class    |
| cus-20009  | atica          | NULL      | 80   | NULL            | NULL           | Poor            |
| cus-20010  | tanema         | NULL      | 600  | NULL            | NULL           | Rich            |

- We can create a customerTag trigger that automatically checks the bill value and set the customer status.



# Before Insert Trigger

- delimiter &&
- mysql> create trigger customerTag  
before insert on customer for each row  
begin  
if new.bill>=500 then set new.customer\_status="Rich";  
elseif (new.bill>=200 and new.bill<500) then set  
new.customer\_status="Middle\_class";  
else set new.customer\_status="Poor";  
end if;  
end&&
- Now check the trigger status as  
insert into customer (customerNo,customerName,bill) values ("cus-  
20007","lily",600);

# After Insert Trigger

- It is activated after the insertion of data into the table.
- Consider the following two tables employee and employees\_audit
- We create a after insert trigger that insert automatically a new employee information to employee\_audit table

```
mysql> desc employee;
-> &&
```

| Field       | Type        | Null | Key | Default | Extra |
|-------------|-------------|------|-----|---------|-------|
| employee_id | int         | NO   | PRI | NULL    |       |
| first_name  | varchar(45) | YES  |     | NULL    |       |
| last_name   | varchar(45) | YES  |     | NULL    |       |
| hire_date   | date        | YES  |     | NULL    |       |

```
mysql> desc employee_audit;
-> &&
```

| Field       | Type        | Null | Key | Default           | Extra             |
|-------------|-------------|------|-----|-------------------|-------------------|
| audit_id    | int         | NO   | PRI | NULL              | auto_increment    |
| employee_id | int         | YES  |     | NULL              |                   |
| action      | varchar(45) | YES  |     | NULL              |                   |
| action_date | timestamp   | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |

# After Insert Trigger

- -- Create the employees table

```
CREATE TABLE employee (  
    employee_id INT PRIMARY KEY,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    hire_date DATE  
);
```

- -- Create the employee\_audit table

```
CREATE TABLE employee_audit (  
    audit_id INT PRIMARY KEY AUTO_INCREMENT,  
    employee_id INT,  
    action VARCHAR(50),  
    action_date TIMESTAMP DEFAULT  
    CURRENT_TIMESTAMP  
);
```

- delimiter &&

- mysql>

```
CREATE TRIGGER after_employee_insert  
AFTER INSERT ON employee  
FOR EACH ROW  
BEGIN
```

```
    INSERT INTO employee_audit (employee_id, action)  
    VALUES (NEW.employee_id, 'INSERT');
```

```
END$$
```

- **Check trigger** as insert into employee table

- INSERT INTO employees (employee\_id, first\_name, last\_name, hire\_date) VALUES (1, 'John', 'Doe', '2023-10-01');

- -- Check the employee\_audit table

- SELECT \* FROM employee\_audit;

# Before Update Trigger

- It is activated before the update of data in the database table.
- Consider the following two tables products and price\_change\_log
- We create a before update trigger that insert into a new record into price\_change\_log table before updating any record into the products table.

```
mysql> desc products;
-> &&
```

| Field        | Type          | Null | Key | Default | Extra |
|--------------|---------------|------|-----|---------|-------|
| product_id   | int           | NO   | PRI | NULL    |       |
| product_name | varchar(100)  | YES  |     | NULL    |       |
| price        | decimal(10,2) | YES  |     | NULL    |       |

```
mysql> desc price_change_log;
-> &&
```

| Field       | Type          | Null | Key | Default           | Extra             |
|-------------|---------------|------|-----|-------------------|-------------------|
| log_id      | int           | NO   | PRI | NULL              | auto_increment    |
| product_id  | int           | YES  |     | NULL              |                   |
| old_price   | decimal(10,2) | YES  |     | NULL              |                   |
| new_price   | decimal(10,2) | YES  |     | NULL              |                   |
| change_date | timestamp     | YES  |     | CURRENT_TIMESTAMP | DEFAULT_GENERATED |

# Before Update Trigger

- -- Create the products table

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100),  
    price DECIMAL(10, 2)  
);  
  
• -- Create the price_change_log table  
CREATE TABLE price_change_log (  
    log_id INT PRIMARY KEY AUTO_INCREMENT,  
    product_id INT,  
    old_price DECIMAL(10, 2),  
    new_price DECIMAL(10, 2),  
    change_date TIMESTAMP DEFAULT  
    CURRENT_TIMESTAMP  
);
```

```
CREATE TRIGGER before_price_update  
BEFORE UPDATE ON products  
FOR EACH ROW  
BEGIN  
    -- Ensure the new price is not negative  
    IF NEW.price < 0 THEN  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Price cannot be negative';  
    END IF;  
    -- Log the price change  
    IF OLD.price != NEW.price THEN  
        INSERT INTO price_change_log (product_id, old_price,  
        new_price)  
        VALUES (OLD.product_id, OLD.price, NEW.price);  
    END IF;  
END$$
```

# Before Update Trigger

- Now check the trigger using the following commands

- -- Insert a record into product table

```
INSERT INTO products (product_id, product_name, price)
VALUES (1, 'Laptop', 1200.00);
```

- -- Update the product's price

```
UPDATE products
SET price = 1300.00
WHERE product_id = 1;
```

- -- Check the price\_change\_log table

```
SELECT * FROM price_change_log;
```

- -- Try to set a negative price (this will fail)

```
UPDATE products
SET price = -200.00
WHERE product_id = 1;
```

# After Update Trigger

- It is activated after the update of data in the database table.
- Consider the following two tables employees and employees\_audit
- We create a after update trigger that insert into a new record into employees\_audit table after updating any record into the employees table.

|                                                                                                                                                |                                                                                                                                                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| • CREATE TABLE employees (<br>employee_id INT PRIMARY KEY,<br>first_name VARCHAR(50),<br>last_name VARCHAR(50),<br>salary DECIMAL(10, 2)<br>); | CREATE TABLE employees_audit (<br>audit_id INT AUTO_INCREMENT PRIMARY KEY,<br>employee_id INT,<br>old_salary DECIMAL(10, 2),<br>new_salary DECIMAL(10, 2),<br>updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP<br>); |
|------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# After Update Trigger

- DELIMITER &&
- CREATE TRIGGER after\_employee\_update  
AFTER UPDATE ON employees  
FOR EACH ROW  
BEGIN  
    -- Check if the salary has changed  
    IF OLD.salary != NEW.salary THEN  
        INSERT INTO employees\_audit (employee\_id, old\_salary, new\_salary)  
        VALUES (OLD.employee\_id, OLD.salary, NEW.salary);  
    END IF;  
END&&
-



# After Update Trigger check

- Now check the trigger using the following commands

Update employees table

```
update employees set salary=45000.00 where employee_id=1;
```

check employees table

```
select * from employees;
```

check employee\_audit table

```
select * from employee_audit;
```

# Before Delete Trigger

- It is activated before the data is removed from the database table.
- Consider the following two tables employees and deleted\_employees\_log table
- We create a before delete trigger that insert into a new record into deleted\_employees\_log table before deleting any record into the employees table.

|                                                                                                                                                                                                                      |                                                                                                                                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• CREATE TABLE employees (<br/>    employee_id INT PRIMARY KEY,<br/>    first_name VARCHAR(50),<br/>    last_name VARCHAR(50),<br/>    department VARCHAR(50)<br/>);</li></ul> | <ul style="list-style-type: none"><li>• CREATE TABLE deleted_employees_log (<br/>    log_id INT AUTO_INCREMENT PRIMARY KEY,<br/>    employee_id INT,<br/>    deleted_at TIMESTAMP DEFAULT<br/>    CURRENT_TIMESTAMP<br/>);</li></ul> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

# Before Delete Trigger

- DELIMITER &&

```
CREATE TRIGGER before_employee_delete
BEFORE DELETE ON employees FOR EACH ROW
BEGIN
    INSERT INTO deleted_employees_log (employee_id)
    VALUES (OLD.employee_id);
END;
END&&
```

# Before Delete Trigger

- Now check the trigger using the following commands
- Insert few data on employees table
- delete from employees where employee\_id=3;
- 
- check data on deleted\_employees\_log

# After Delete Trigger

- It is activated after the data is removed from the database table.
- Consider the following two tables employees and deleted\_employees\_log table
- We create a before delete trigger that insert into a new record into deleted\_employees\_log table after deleting any record into the employees table.

```
• CREATE TABLE employees (  
  employee_id INT PRIMARY KEY,  
  employee_name VARCHAR(100),  
  department VARCHAR(100)  
);
```

```
• CREATE TABLE employee_audit (  
  audit_id INT PRIMARY KEY AUTO_INCREMENT,  
  employee_id INT,  
  employee_name VARCHAR(100),  
  department VARCHAR(100),  
  deleted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

# After Delete Trigger

- DELIMITER &&

```
CREATE TRIGGER after_employee_delete
```

```
AFTER DELETE ON employees FOR EACH ROW
```

```
BEGIN
```

```
    INSERT INTO employee_audit (employee_id, employee_name, department)
```

```
VALUES (OLD.employee_id, OLD.employee_name, OLD.department);
```

```
END;
```

```
END&&
```

# After Delete Trigger

- Now check the trigger using the following commands
- Insert few data on employees table
- `delete from employees where employee_id=2;`
- check data on employees\_audit



**THANK YOU**